

# Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللهی بیوکی

Saeed Amrollahi Boyouki

[www.Linkedin.com/in/saeed-amrollahi-boyouki](http://www.Linkedin.com/in/saeed-amrollahi-boyouki)

# Agenda 15/30

## Introduction to contiguously sequential data structures: C-Style strings and C++ strings

### Part 4

- The C-Style string and String literals
- C++ standard library: The string class
- String manipulation: writing simple programs
- References and further readings
- Q & A

120 min (incl. Q & A)



# String literals

- A *string literal* is a character sequence enclosed within double quotes:

`"this is a string"`

|   |   |   |   |  |   |   |  |   |  |   |   |   |   |   |   |    |
|---|---|---|---|--|---|---|--|---|--|---|---|---|---|---|---|----|
| t | h | i | s |  | i | s |  | a |  | s | t | r | i | n | g | \0 |
|---|---|---|---|--|---|---|--|---|--|---|---|---|---|---|---|----|

```
char v[17];      char *v = "this is a string";
```

- The type of a string literal is "array of the appropriate number of const characters."
- A string literal can be assigned to a `char*`.

```
char* p = "Hello";  
"" // empty string: const char[1]
```

```
wchar_t* p = L"Hello";  
L"" // empty string: const wchar_t[1]
```

- Escape characters:

| Name           | ASCII name | C++ name |
|----------------|------------|----------|
| newline        | NL (LF)    | \n       |
| horizontal tab | HT         | \t       |
| vertical tab   | VT         | \v       |
| backspace      | BS         | \b       |
| alert          | BEL        | \a       |
| question mark  | ?          | \?       |
| single quote   | '          | \'       |
| double quote   | "          | \"       |

# Writing simple functions

```
/* atoi: convert s to integer */
int atoi(const char s[])
{
    int i, n;
    n = 0;
    for (i = 0; s[i] >= '0' && s[i] <= '9'; ++i)
        n = 10 * n + (s[i] - '0');

    return n;
}
```

```
/* strlen: return length of string s */
int strlen(const char *s)
{
    int n;
    for (n = 0; *s != '\0'; s++)
        n++;

    return n;
}
```

# Writing simple functions cont.

```
/* strcpy: copy t to s; array subscript version */
void strcpy(char *s, const char *t)
{
    int i;
    i = 0;
    while ((s[i] = t[i]) != '\0')
        i++;
}
```

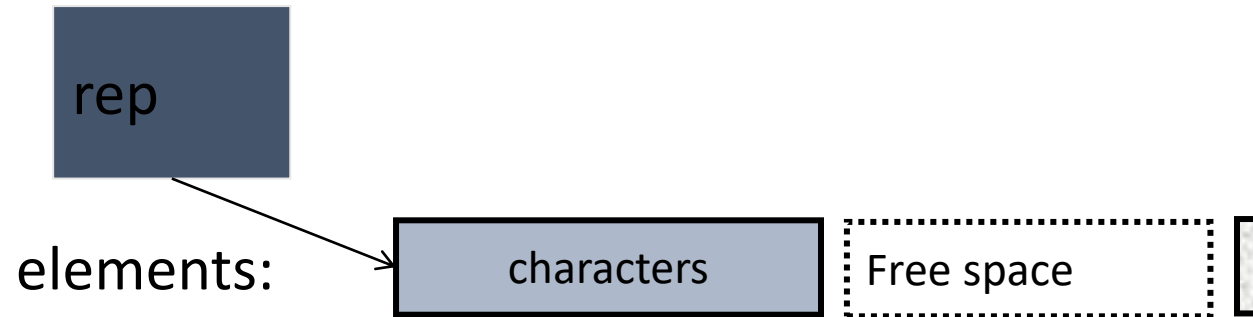
```
/* strcpy: copy t to s; pointer version */
void strcpy(char *s, char *t)
{
    int i;
    i = 0;
    while ((*s = *t) != '\0') {
        s++;
        t++;
    }
}
```

- String concatenation

  
*Strcat Test*  
Prog.

# String

- A string might be implemented:
  - For short strings that characters are stored in the string handle itself, and
  - For longer strings the elements are stored contiguously on the free-store (like vector elements)



# String manipulation: writing simple classes

- String concatenation

 *First And Last Names* 

- String comparison

 *Compare Two Strings* 

- Delete repeated words

 *Delete Repeated Words* 

C-style **S**tring literal vs. string

# C-style **S**tring literal vs. string

C-style character array

C++ standard string

# C-style **S**tring literal vs. string

C-style character array

From day one

C++98

C++ standard string

# C-style **S**tring literal vs. string

C-style character array

From day one

Zero-terminated string literal

std::string object

C++98

C++ standard string

# C-style **S**tring literal vs. string

## C-style character array

From day one

Zero-terminated string literal

The language core

Standard library: `<string>`

`std::string` object

C++98

C++ standard string

# C-style **S**tring literal vs. string

## C-style character array

From day one

Zero-terminated string literal

The language core

Compile-time constant size

Run-time variable size

Standard library: `<string>`

`std::string` object

C++98

C++ standard string

# C-style **S**tring literal vs. string

## C-style character array

From day one

Zero-terminated string literal

The language core

Compile-time constant size

Size unaware

[ ], pointer arithmetic

[ ]

Size-aware

Run-time variable size

Standard library: <string>

std::string object

C++98

C++ standard string

# C-style **S**tring literal vs. string

## C-style character array

From day one

Zero-terminated string literal

The language core

Compile-time constant size

Size unaware

[ ], pointer arithmetic

Automatic storage

Fixed size/no  
grow and shrink

Implicit decay of name  
to pointer/pointer  
arithmetic

No implicit decay of  
name to pointer/pointer  
arithmetic

Variable size/grow  
and shrink

Dynamic storage

[ ]

Size-aware

Run-time variable size

Standard library: <string>

std::string object

C++98

C++ standard string

# C-style array of chars vs. standard library string : comparison

- char string is defined in core language, but string is defined in standard library.
- The size of char string can be determined using null character/strlen() function, but the size of string can be determined using size() member function.
- char string is not generic, but standard string are generic.
- In array of char, we may need to use pointer arithmetic but in string we don't.
- In array of char, we may need to use memory management but in string we don't.



Use string rather than zero-terminated arrays of char.

# R references and further readings

- [Str00] Bjarne Stroustrup. The C++ Programming Language (Special Edition). Addison Wesley. Reading Mass. USA. February 2000.
- [Str08] Bjarne Stroustrup. Programming: Principles and Practice in C++, 1<sup>st</sup> edition, Addison Wesley, December 2008.
- [Str13] Bjarne Stroustrup. The C++ Programming Language. 4<sup>th</sup> edition. Addison Wesley. Reading Mass. USA. May 2013.
- [Str22] Bjarne Stroustrup. A Tour of C++, 3<sup>rd</sup> edition, Addison Wesley, September 2022.

*Thanks for your patience ...*

A man who asks a question is a fool for minute,  
The man who does not ask, is a fool for a life.  
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.  
- Howard Hinnant

